Data Indexing and Selection

Python Data Science Handbook — Chapter 3, Section 3.2

Getting data *out* of Series and DataFrames — and the label-vs-position trap that `loc` and `iloc` were built to defuse.

1 Why this section deserves its own chapter slot

In section 3.1 we built `Series` and `DataFrame` objects. The obvious next question is the most practical one in all of data work: *how do I pull out the piece I want?* A single value, a column, a block of rows, every row that satisfies a condition.

Background & Context

You already know two indexing vocabularies from your first year of Python, and Pandas deliberately speaks *both at once*:

- **The dictionary vocabulary:** `d[key]` looks something up by a meaningful label. Order is irrelevant; the key is what matters.
- **The array/list vocabulary:** `a[3]`, `a[1:4]`, `a[mask]` address data by integer *position* and support slices, boolean masks, and fancy indexing.

A `Series` carries an explicit index (labels) on top of an implicit one (integer positions). So both vocabularies apply to the same object — and when the labels themselves happen to be integers, the two collide. Most of this section is about understanding that collision and the tools (`loc`, `iloc`) that make your intent unambiguous.

2 Series as a dictionary

The cleanest mental model for a `Series` is a dictionary whose keys are kept in a typed, ordered `Index`. Let us build a tiny one — the land area, in thousands of square kilometers, of four U.S. states:

```
import pandas as pd

area = pd.Series({'California': 424, 'Texas': 696,
                  'Florida': 170, 'NewYork': 141})
area['Texas']        # → 696
```

Every dictionary idiom you know carries over. You can test membership, list the keys, and iterate over key–value pairs:

```
'Texas' in area      # → True
'Ohio' in area       # → False
area.keys()          # → Index(['California','Texas','Florida','NewYork'],
   ...)
list(area.items())   # → [('California', 424), ('Texas', 696), ...]
```

Python Refresher

`.items()` mirrors `dict.items()`: it yields `(label, value)` tuples, which is exactly what you would iterate over in a `for` loop. The membership operator `in` checks the *index* (the keys), not the values — just as `'x' in some_dict` checks keys. If you want to ask whether a *value* occurs, use something like `(area == 696).any()` instead.

A `Series` is also mutable in the dictionary sense: assigning to a brand-new key extends it in place, growing the index by one entry.

```
area['Ohio'] = 116
print(area)
```

```
California    424
Texas         696
Florida       170
NewYork       141
Ohio          116
dtype: int64
```

Key Idea

This dictionary-like flexibility is what separates a `Series` from a raw NumPy array. A NumPy array has a fixed size and an implicit integer index you cannot rename; a `Series` lets you *add* keys, look up by meaningful labels, and still keep the underlying values in a fast NumPy buffer.

3 Series as a one-dimensional array

The same object also behaves like a NumPy array: it supports slices, boolean masks, and fancy indexing. Here is where the subtlety begins. Reset to a small alphabetic index so we can see both index systems clearly:

```
data = pd.Series([0.25, 0.50, 0.75, 1.00],
                 index=['a', 'b', 'c', 'd'])
```

3.1 Two kinds of slice, two different rules

```
data['a':'c']   # slice by EXPLICIT label
data[0:2]       # slice by IMPLICIT integer position
```

```
# data['a':'c']
a   0.25
b   0.50
c   0.75

# data[0:2]
```

```
a   0.25
b   0.50
```

Look closely. The label slice `'a':'c'` returns *three* elements — it **includes** the endpoint `'c'`. The positional slice `0:2` returns *two* elements — it **excludes** position 2, the ordinary Python "stop is exclusive" rule.

Common Pitfall

The single most confusing thing in all of Pandas. Label-based slices are *inclusive* of the final label; integer-position slices are *exclusive* of the final position. Two opposite conventions live inside one object. The reason is sensible — with labels, you often do not know or care what position `'c'` sits at, so "up to and including `'c'`" is the intuitive meaning — but it means you must always know *which* kind of slice you are writing.

Boolean masking and fancy indexing work just like NumPy:

```
data[(data > 0.3) & (data < 0.8)]   # boolean mask
data[['a', 'd']]                    # fancy indexing
```

```
# masked
b   0.50
c   0.75

# fancy
a   0.25
d   1.00
```

3.2 The integer-index disaster, and how `loc`/`iloc` fix it

Now make the labels themselves integers — but *not* the usual `0,1,2...`:

```
weird = pd.Series(['a', 'b', 'c'], index=[1, 3, 5])
weird[1]      # explicit-index lookup → 'a'  (label 1)
weird[1:3]    # implicit-position slice → labels 3 and 5
```

```
weird[1]
'a'

weird[1:3]
3    b
5    c
dtype: object
```

Notice the schizophrenia: when you *index* with a single integer, Pandas uses the *explicit label* (so `weird[1]` is the element labelled 1); but when you *slice* with integers, Pandas falls back to *implicit position* (so `weird[1:3]` means positions 1 and 2). Same brackets, two different index systems, chosen by whether you passed a scalar or a slice. This is a bug factory.

Pandas resolves it with two explicit accessors:

- `.loc[...]` *always* uses the **explicit label**, for both indexing and slicing (and label slices stay endpoint-inclusive).
- `.iloc[...]` *always* uses the **implicit integer position**, NumPy-style (endpoint-exclusive).

```
weird.loc[1]       # → 'a'        (label 1)
weird.loc[1:3]     # → labels 1 and 3, INCLUSIVE
weird.iloc[1]      # → 'b'        (position 1)
weird.iloc[1:3]    # → positions 1 and 2, EXCLUSIVE
```

Key Idea

The Zen of Python says *"explicit is better than implicit."* That line is practically the motto of `loc` and `iloc`. Bare-bracket indexing on a `Series` guesses your intent from context; `loc` and `iloc` never guess. In production code and shared notebooks, reach for them by default — they make your code read the same way it behaves, and they are immune to the integer-label trap above.

For grabbing or setting a *single scalar* as fast as possible, Pandas also offers `.at` (label-based) and `.iat` (position-based). They skip the machinery that lets `loc`/`iloc` return slices and masks, so `df.at['Texas', 'area']` is a touch quicker than `df.loc['Texas', 'area']` when you truly want exactly one cell.

4 DataFrame as a dictionary of Series

Step up one dimension. Build a small table by pairing our `area` with a population `Series`, then derive a density column:

```
pop = pd.Series({'California': 39.5, 'Texas': 29.0,
                 'Florida': 21.5, 'NewYork': 19.5})   # millions
states = pd.DataFrame({'pop': pop, 'area': area})
states['density'] = states['pop'] / states['area']     # people per 1000 km
    ^2
```

	pop	area	density
California	39.5	424	0.0932
Texas	29.0	696	0.0417
Florida	21.5	170	0.1265
NewYork	19.5	141	0.1383

Intuition

Read a `DataFrame` as a dictionary whose *keys are column names* and whose *values are* `Series`. That is why a single bracketed key returns a column: `states['area']` hands you the `area` `Series`, not a row. (Coming from NumPy this surprises people, because there `a[0]` is a row.

Hold that thought — the convention flips for slices.)

There are two ways to pull out a column:

```
states['area']    # dictionary-style: always works
states.area       # attribute-style: convenient, but fragile
```

Attribute access is sugar that works only when the column name is a valid Python identifier *and* does not collide with an existing `DataFrame` method or attribute.

Common Pitfall

Three ways attribute access bites you.

1. **Name collisions.** A `DataFrame` already has a `.pop()` method (it removes a column). If your column is called `pop`, then `states.pop` returns the *method*, not your data — silently wrong. Bracket access `states['pop']` is unambiguous.
2. **Illegal identifiers.** A column named `'mean temp'` (with a space) or `'2020'` cannot be reached as `states.mean temp`; only `states['mean temp']` works.
3. **You cannot create a column by attribute.** `states.newcol = ...` does *not* add a column — it just tacks an ordinary attribute onto the object. To create a column you must assign through brackets: `states['newcol'] = ...`.

The safe habit: *read* with either style, but *write* (and resolve any doubt) with brackets.

5 DataFrame as a two-dimensional array

The other face of a `DataFrame` is an enhanced 2-D NumPy array. The raw grid lives in `.values`, and you can transpose with `.T`:

```
states.values     # the underlying 2-D NumPy array (no labels)
states.T          # swap rows and columns (states become columns)
```

Python Refresher

`.T` is the same transpose idea as NumPy's `a.T` or a math textbook's A^{T}: element (i, j) becomes element (j, i). After `states.T`, the column labels (`pop, area, density`) become the row index and the state names become the columns. Transpose is purely a relabeling-and-reshaping view; it does no arithmetic.

For genuine two-dimensional selection — pick rows *and* columns at once — use `loc` and `iloc` with two arguments, exactly like NumPy's `a[rows, cols]` but with labels available:

```
states.iloc[:2, :2]                 # first 2 rows, first 2 cols (by
    position)
states.loc[:'Texas', :'area']       # up to row 'Texas', up to col 'area'
    (labels, inclusive)
states.loc['Florida', 'density']    # a single labelled cell
```

```
# states.iloc[:2, :2]
pop   area
California   39.5   424
Texas       29.0   696
```

The real power appears when you combine masking (which rows) with fancy indexing (which columns) inside one `loc` call:

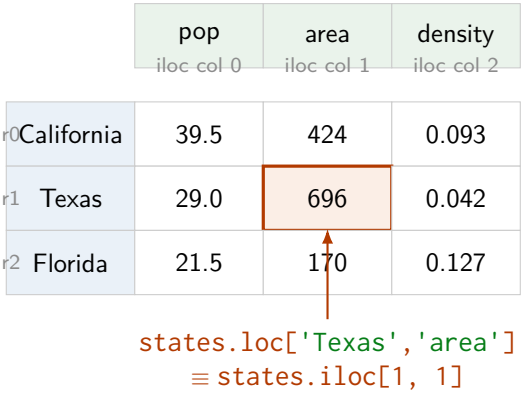
```
states.loc[states['density'] > 0.1, ['pop', 'density']]
```

```
        pop   density
Florida    21.5   0.1265
NewYork   19.5   0.1383
```

Read that left to right: "from `states`, take the rows where density exceeds 0.1, and from those rows keep only the `pop` and `density` columns." This one-liner is the workhorse pattern of exploratory analysis.

5.1 A picture of `loc` versus `iloc`

Both accessors can reach the very same cell; they differ only in the *address system* they use to name it. The diagram below shows our `states` table with its labels (outside, blue/green) and its implicit integer positions (inside each header, gray). The highlighted cell — Texas's area — is reached by `.loc['Texas','area']` or equivalently `.iloc[1, 1]`.



Intuition

`loc` navigates by the *names written on the outside* of the grid (blue rows, green columns); `iloc` navigates by the *counting numbers* `0,1,2...` regardless of what the labels say. Pick `loc` when you think "the Texas row, the area column"; pick `iloc` when you think "the second row, the second column."

6 The conventions that surprise everyone

Bare-bracket indexing on a `DataFrame` (no `loc`/`iloc`) follows a small set of rules that feel inconsistent until you memorize them. They exist because they cover the two most common everyday actions:

"give me a column" and "give me some rows."

```
states['area']          # a bare KEY  → a COLUMN
states['California':'Texas']  # a SLICE → ROWS (label, inclusive)
states[1:3]             # a positional SLICE → ROWS (exclusive)
states[states['density'] > 0.1]  # a MASK → ROWS
```

Key Idea

The rule of thumb for bare brackets on a `DataFrame`:

- A **single key** selects a **column** (dictionary view).
- A **slice** or a **boolean mask** selects **rows** (array view).

So `df['x']` and `df['a':'c']` address *different axes* despite using the same brackets. When this ever feels ambiguous — and it will — stop and rewrite it with `loc` or `iloc`, which name the axis explicitly: `df.loc[:, 'x']` for the column, `df.loc['a':'c', :]` for the rows.

6.1 Chained indexing and the `SettingWithCopyWarning`

A final, very practical trap. Suppose you want to bump Texas's density. The tempting two-step write is:

```
states['density']['Texas'] = 0.05     # chained indexing -- avoid
```

This is *chained indexing*: two bracket operations back to back. The first, `states['density']`, may return either a view onto the original data or a fresh copy — Pandas does not guarantee which. If it returns a copy, your assignment lands on a temporary that is thrown away, and your `DataFrame` is silently *not* updated. To warn you that the outcome is undefined, Pandas raises a `SettingWithCopyWarning`.

Common Pitfall

Never assign through chained brackets. Do the row-and-column selection in a *single* `loc` call, which Pandas guarantees writes back to the original:

- Wrong: `states['density']['Texas'] = 0.05`
- Right: `states.loc['Texas', 'density'] = 0.05`

The single-`loc` form is unambiguous about which cell you mean and is guaranteed to mutate the original frame. (In Pandas 3.0 the underlying copy-on-write behavior makes the old chained form fail more loudly, but the single-`loc` habit is correct under every version.)

Section recap

- A `Series` acts like a dict (`in`, `.keys()`, `.items()`, assign a new key to extend) and like a 1-D array (slices, masks, fancy indexing).
- Label slices include the endpoint; positional slices exclude it. With integer labels, bare brackets index by label but slice by position — a genuine trap.
- `.loc` = explicit labels (inclusive slices); `.iloc` = implicit integer positions (exclusive slices).

`.at`/`.iat` are their fast single-scalar cousins. Prefer them: explicit beats implicit.

- A `DataFrame` is a dict of `Series`: a bare key returns a *column*. Attribute access (`df.col`) is convenient but breaks on method-name collisions, spaces, and cannot create new columns — write with brackets.
- As a 2-D array it offers `.values`, `.T`, and combined row+column selection via `loc`/`iloc`, e.g. `df.loc[df['density'] > 0.1, ['pop','density']]`.
- Bare-bracket surprise: a key → column, but a slice or mask → rows.
- Never assign through chained indexing; use one `loc` assignment to avoid `SettingWithCopyWarning`.